

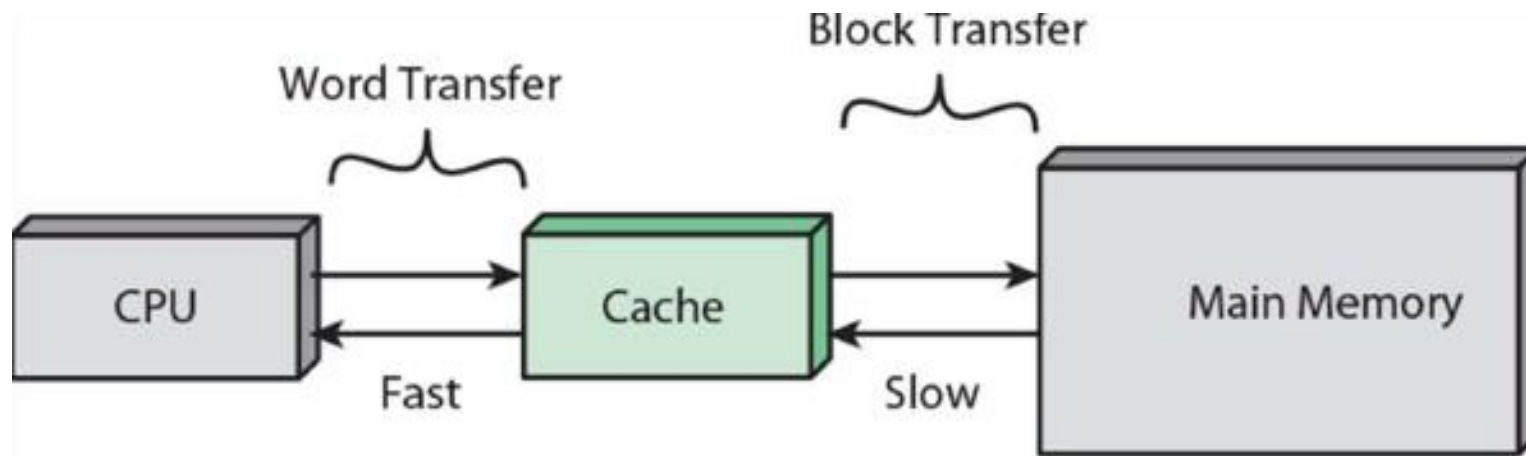
Cache Memory & Mapping Techniques

Module 4

Ref. William Stallings

Cache

- Small amount of fast memory
- Available between main memory and CPU

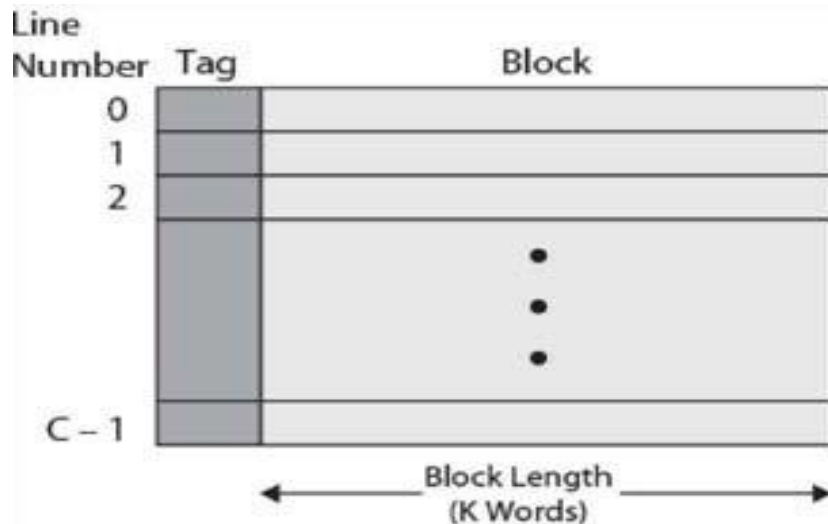


(a) Single cache

Word- is a unit of data with a fixed bit length

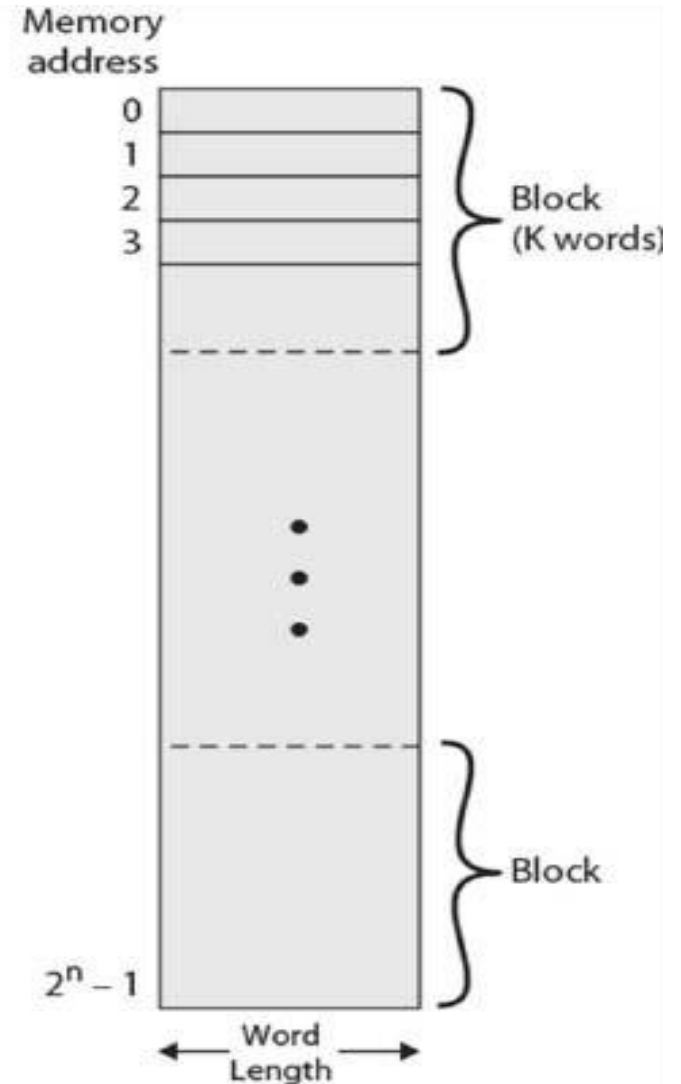
Data block -is a sequence of data that can be transferred as a whole.

Cache/Main Memory Structure



(a) Cache

- A tag in cache memory is a unique identifier that matches a **cache line** to a specific location in main memory.
- It also indicates the status of the cache line.

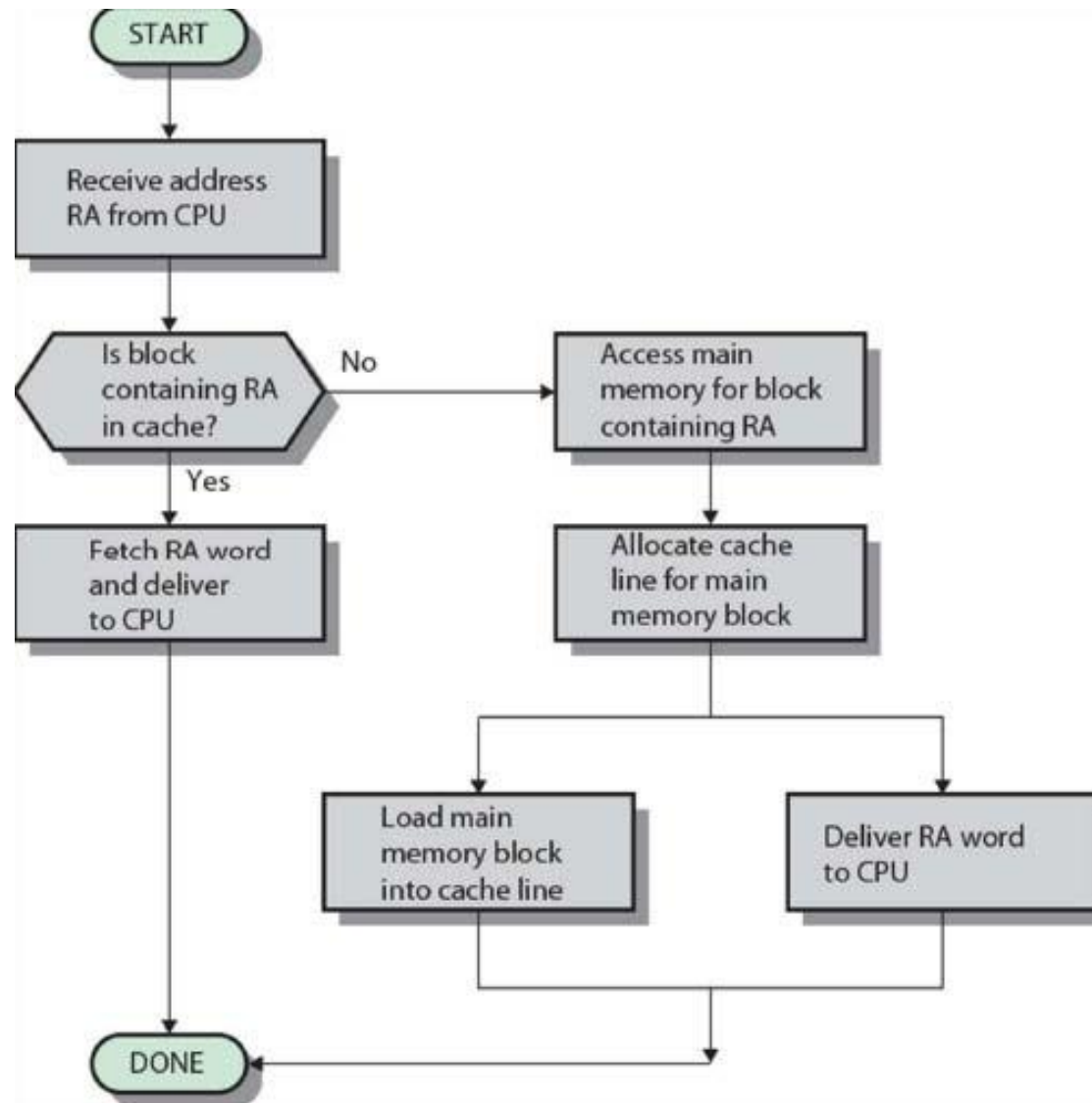


(b) Main memory

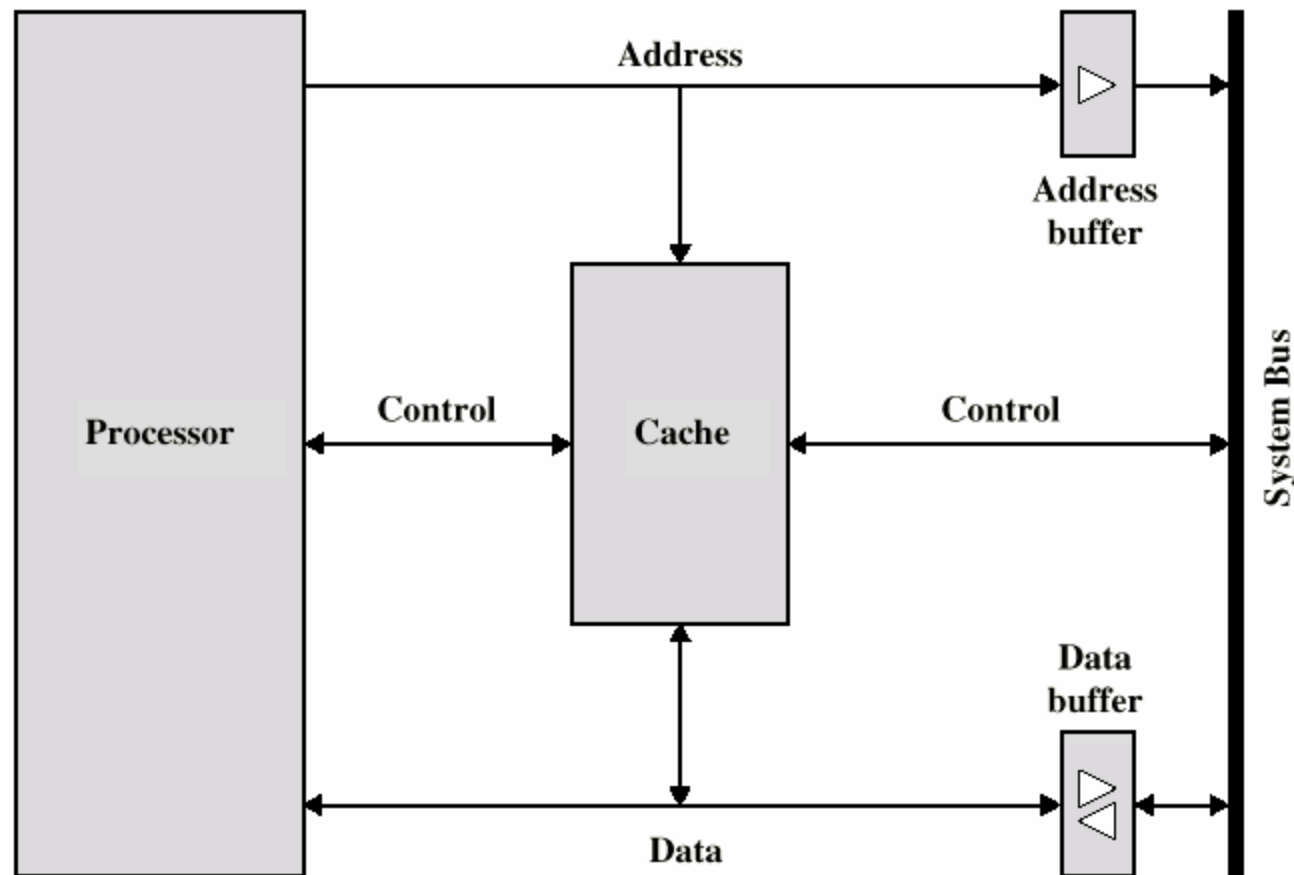
Cache operation – overview

- CPU requests contents of memory location
- Checks cache for this data
- If present, gets from cache (fast)
- If not present, reads required block from main memory to cache and then delivers from cache to CPU
- **Cache includes tags to identify which block of main memory is in each cache line**

Cache Read Operation - Flowchart



Typical Cache Organization



Elements of Cache Design

1. Size
2. Mapping Function
 - 1) associative, 2) direct, 3) set-associative
3. Replacement Algorithm
 - 1) LRU, 2) LFU, 3) FIFO, 4) Random
4. Write Policy
 - 1) Write-through, 2) Write-back
5. Block Size
6. Number of Caches
 - 1) Single or 2-level, 2) Unified or Split

Size

- Cost
 - More cache is expensive
- Speed
 - More cache is faster (up to a point)
 - Checking cache for data takes time

Principle of Locality

A program tends to access a relatively small region of memory irrespective of its actual memory footprint in any given interval of time. While the region of activity may change over time, such changes are gradual.

Temporal Locality: Tendency for a location that has been accessed to be accessed again

Spatial Locality: Tendency for locations close to a location that has been accessed to also be accessed

Basic terminologies- Cache

Hit: A *hit* is when data is found at a given memory level.

Miss: A *miss* is when it is not found in cache.

Hit rate (*h*) is probability of *successful lookup* in cache by CPU.

(or)The **hit rate** is the **percentage of time** data is found at a given memory level.

The **miss rate** is the percentage of time it is not in cache.

(Miss rate = 1 - hit rate).

The **miss penalty** is the time required to process a miss, including the time that it takes to replace a block of memory plus the time it takes to deliver the data to the processor.

Hit ratio – Ratio of number of hits to total number of references – number of hits/(number of hits + number of Miss)

Effective Memory Access Time (EMAT): Effective access time experienced by the CPU when accessing memory.

Time to lookup cache to see if memory location is already there

Upon cache miss, time to go to deeper levels of memory hierarchy

$$EMAT = T_c + m * T_m$$

where m is cache miss rate, T_c the cache access time and T_m the miss penalty

Cache Design

If memory contains 2^n addressable words

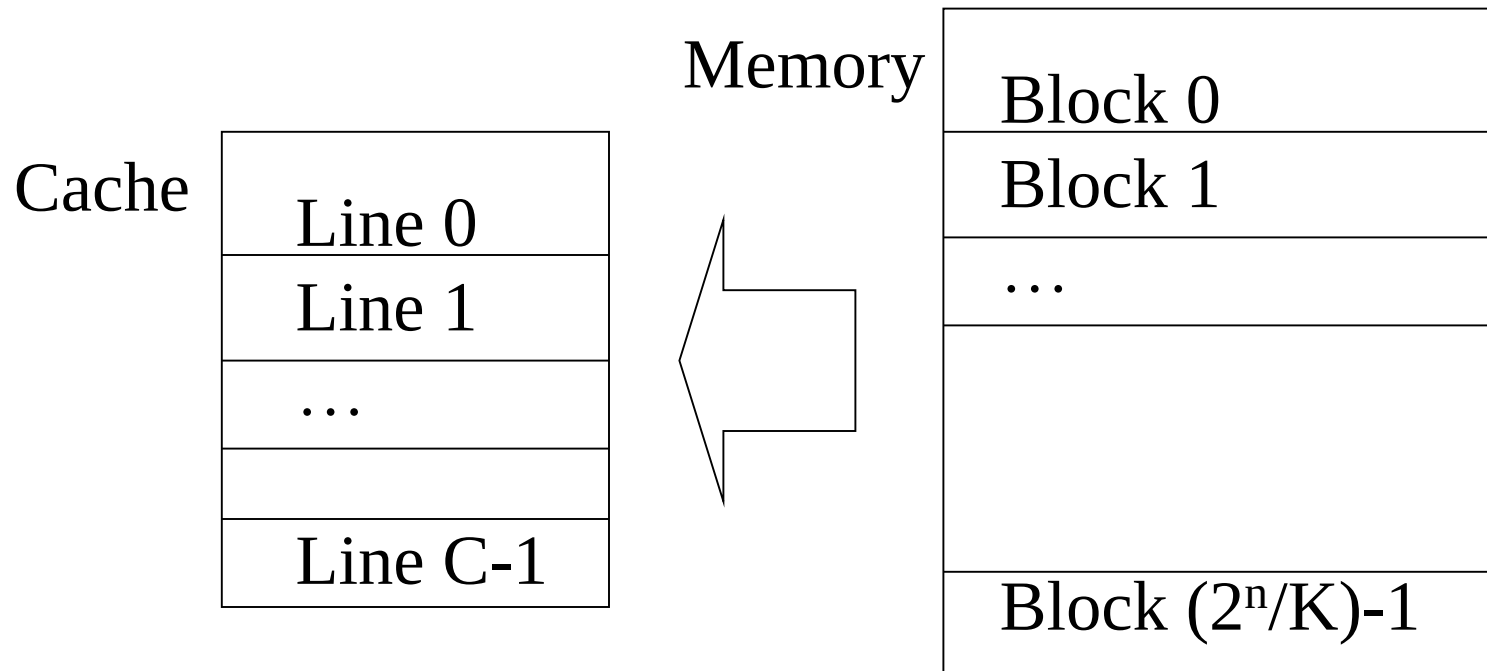
Memory can be broken up into blocks with K words per block.

Number of blocks = $2^n / K$

Cache consists of C **lines** or **slots**, each consisting of K words

$C \ll M$ (*Size of Cache is lesser than Main Memory*)

How to map blocks of memory to lines in the cache?



Need for Mapping Function

As there are **fewer cache lines than memory blocks**, an **algorithm to map main memory block to cache line** is essential.

Also, a means to determine **which MM block is currently in the Cache block** is required.

Mapping Function

Consider the following configuration

Word size is 1 byte

Cache line or Block size is **2 bytes** = 2^1

Cache of 16 bytes with *2 bytes per line*

i.e. cache is $16/2 = 8$ lines i.e. $2^4 / 2^1 = (2^3) = 8$

Will need 8 addresses

Main memory of 64 bytes with *2 bytes per line*

6 bit address needed to reference 64 bytes ($2^6 = 64$)

64 bytes / 2 bytes-per-block

$2^6 / 2^1 = (2^5) = 32$ Memory Blocks

It is required to map the 32 memory blocks to the 8 lines in the cache.

Multiple memory blocks will have to be mapped to the same line in the cache.

Mapping Function – 64K Cache Example

Consider the following configuration

Word size of 1 byte

Cache of 64 KB, with Block size of 4 bytes (2^2)

Cache line or Block size is 4 bytes

i.e. cache is $64 \text{ KB} / 4 \text{ bytes} = 2^{16}/2^2 = (2^{14})$ i.e, 16,384 lines of 4 bytes

Main memory of 16MB

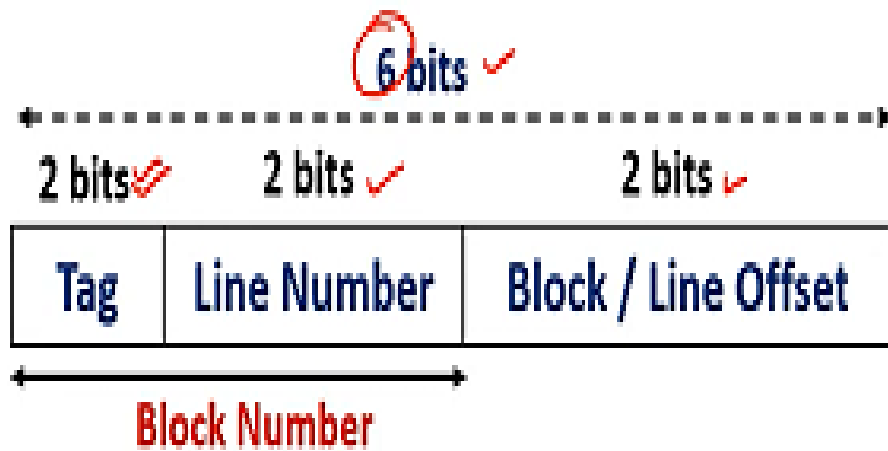
24 bit address = $2^{24}=16\text{MB}$

$16\text{MB} / 4\text{bytes-per-block} = 2^{24}/2^2 \rightarrow 2^{22} = 4 \text{ Mega bytes}$ of Memory Blocks

4 Mega of blocks in memory is mapped onto the 16K of lines in the cache.

Multiple memory blocks will have to map to the same line in the cache.

Division of Physical Address



Total words: 64 → 2^6

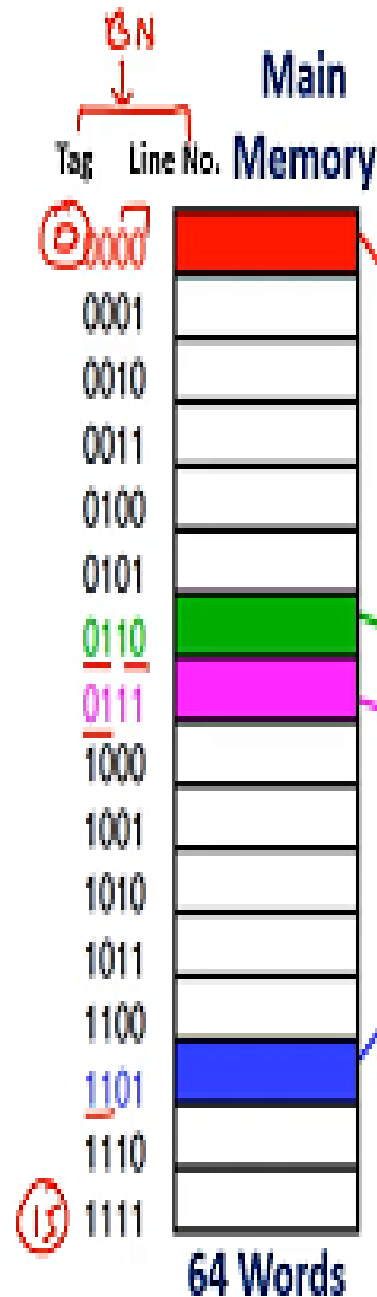
Block Size: 4 → 2^2

Total Line No's: 4 → 2^2

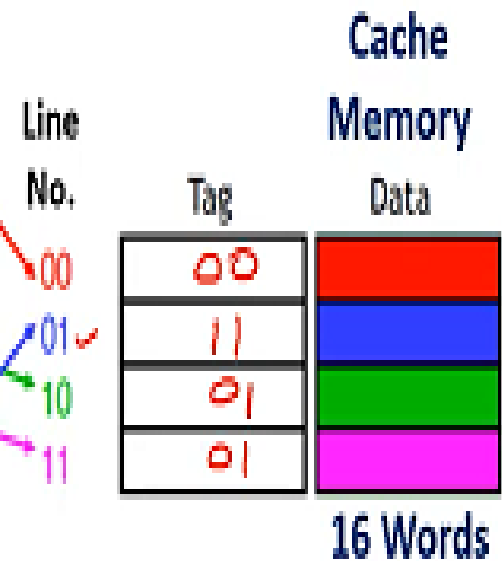
Total Blocks: 16 → 2^4 BN

Example: Physical Address - 00 00 01

Tag	Line Number	Block Offset
-----	-------------	--------------



Block No.	Tag	Line No.
0000	00	00
0001	00	01
0010	00	10



Mapping Function

- Specification of correspondence between main memory blocks and cache blocks
- The transformation of data from main memory to cache memory
 - 1) Associative mapping
 - 2) Direct mapping
 - 3) Set-associative mapping

Mapping Function-Block Placement Strategies

Cache organizations:

Direct mapped: Each block (a small, contiguous chunk of data that is stored as a single unit within the cache) **has only one place in the cache.**

Mapping: **(Block address) MOD (Number of block in cache)**

Fully associative/Associate: A MM block can be placed anywhere in the cache.

Set associative: A block can be placed in a **restricted set of places in the cache**

A set is a group of blocks in the cache.

Mapping: **(Block address) MOD (Number of sets in the cache)**

If there are n blocks in a set, the cache placement is called n-way set associative.

Example

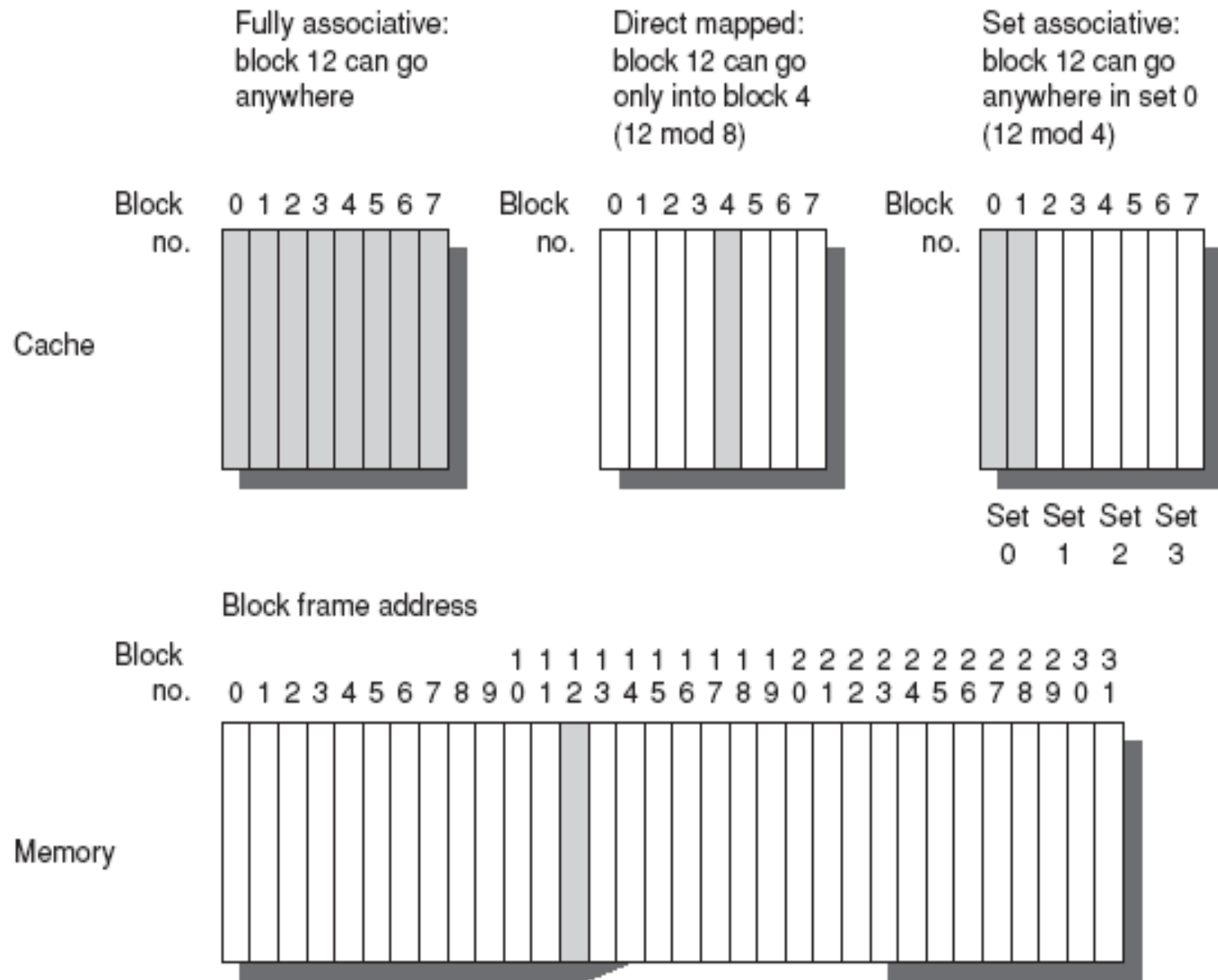


Figure C.2 This example cache has eight block frames and memory has 32 blocks.

Direct Mapping

- Each block of main memory **maps to only one cache line**
- Address consists of **two parts**
 - Least Significant **w bits** - identify unique word
 - Most Significant **s bits** - specify one memory block
- Most Significant **s bits** are split into (in cache)
 - a **cache line** field **r bits** and
 - a **tag** of **s-r bits** (most significant)

Direct Mapping

Simplest mapping technique: each block of main memory maps to only **one specific cache line**

i.e. if a block is in cache, it must be in one specific place

Formula to map a memory block to a cache line:

$$i = j \bmod m$$

i=Cache Line Number

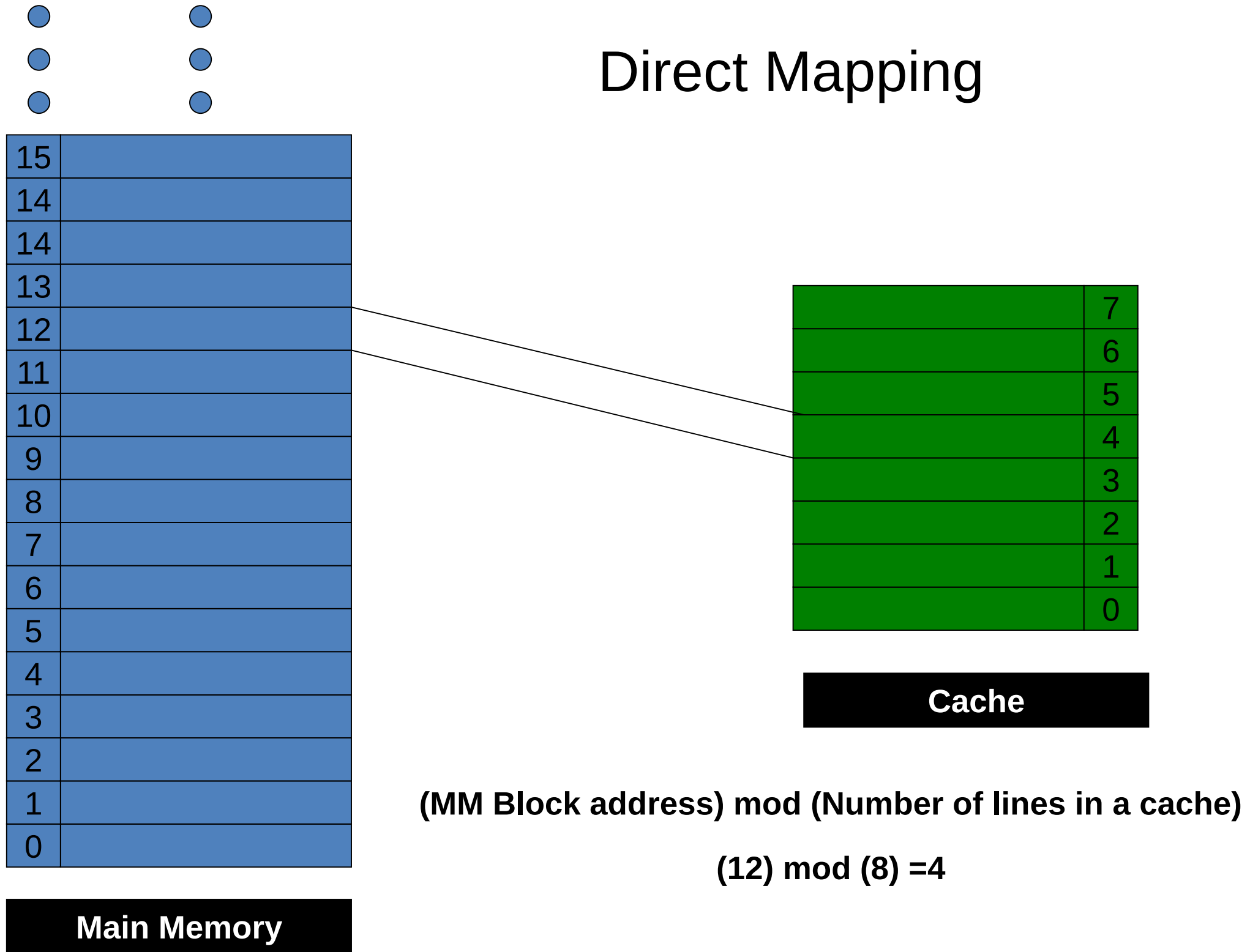
j=Main Memory Block Number

m = Number of Lines in Cache

32 mod 8 = 0 (line 0 of cache) ; 31 mod 8 = 7 (line 7 of cache)

i.e. we divide the memory block by the number of cache lines and **the remainder is the cache line address**

Direct Mapping

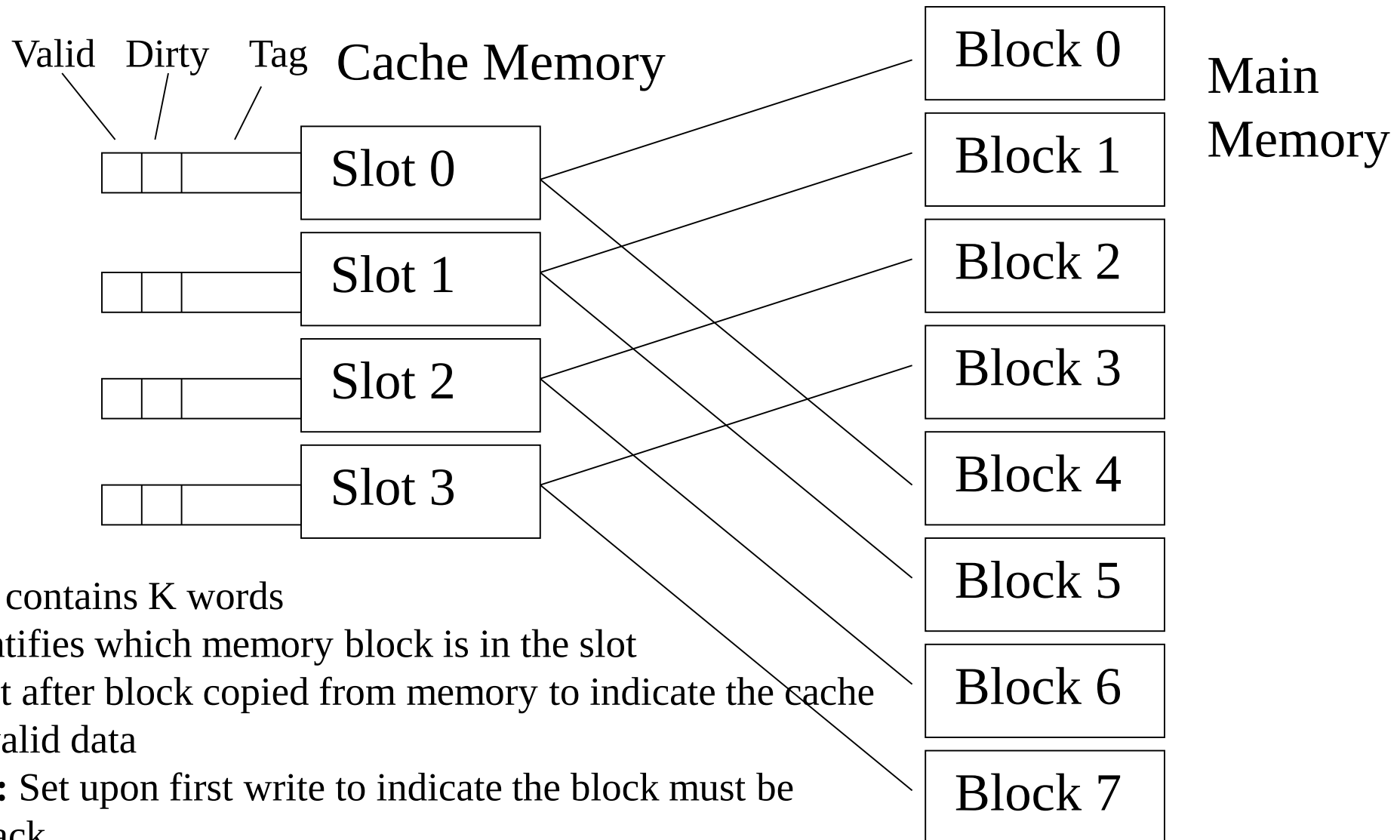


Direct Mapping with $C=4$

Shrinking our example to a cache line size of 4 slots (each slot/line/block still contains 4 words):

Cache Line	Memory Block Held
0	0, 4, 8, ...
1	1, 5, 9, ...
2	2, 6, 10, ...
3	3, 7, 11, ...

Direct Mapping with $C=4$



Direct Mapping Address Structure

Easy way to compute

$i = j \bmod m$ is based upon the bits in the address we are trying to access

Address is in three parts: MM is 256 bytes and block size is 4 bytes. CM is 64 bytes with line size as 4 bytes.

Least Significant **w bits identify unique word within a cache line**

w must be enough bits to address a **specific word in a cache line**;

e.g. if 4 words per cache line, then we need 2 bits for **w**

Next Significant **s bits** specify which line this address maps into

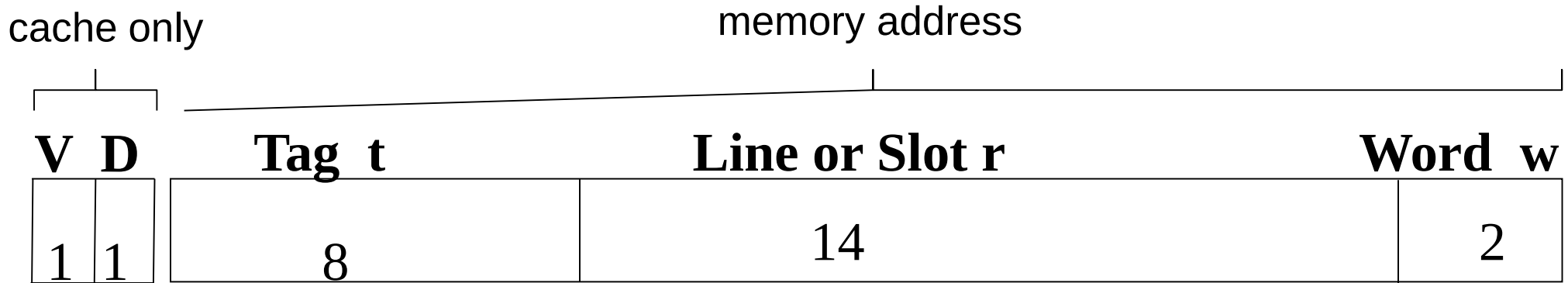
r must be enough bits to address a specific line in the cache; e.g. if

16 cache lines (64/4), then we need 4 bits for **r**

Remaining **t bits** is used as a tag to identify the main memory block in cache

Tag t	Line or Slot r	Word w
2	4	2
8 bit address		

Direct Mapping Address 64KB Cache Example



Given 16MB MM which requires **24 bit address to access**

4 byte block requires 2 bit word identifier ; Therefore 22 MM blocks ($2^{24}/2^2 = 2^{22}$)

64KB (2^{16}) cache requires 16 bit address. If each line is of 4 bytes (2^2), it requires 14 bits to address the cache slot/line

8 bits left for tag (22-14)

No two blocks in the same line have the same Tag field

Valid bit and a Dirty bit

Valid – Indicates if the slot holds a **block belonging to the program being executed**

Dirty – Indicates if a block has been modified while in the cache, meaning that it **is required to be written back to memory** before slot is reused for another block

Direct Mapping Example (64KB Cache)

Cache Memory

	Addr	Tag	W0	W1	W2	W3
Line 0	0	00	F1	F2	F3	F4
Line 1	1	1B	11	12	13	14
	2					
	3					
	4					
	5					
	..					
	..					
	$2^{14}-1$					

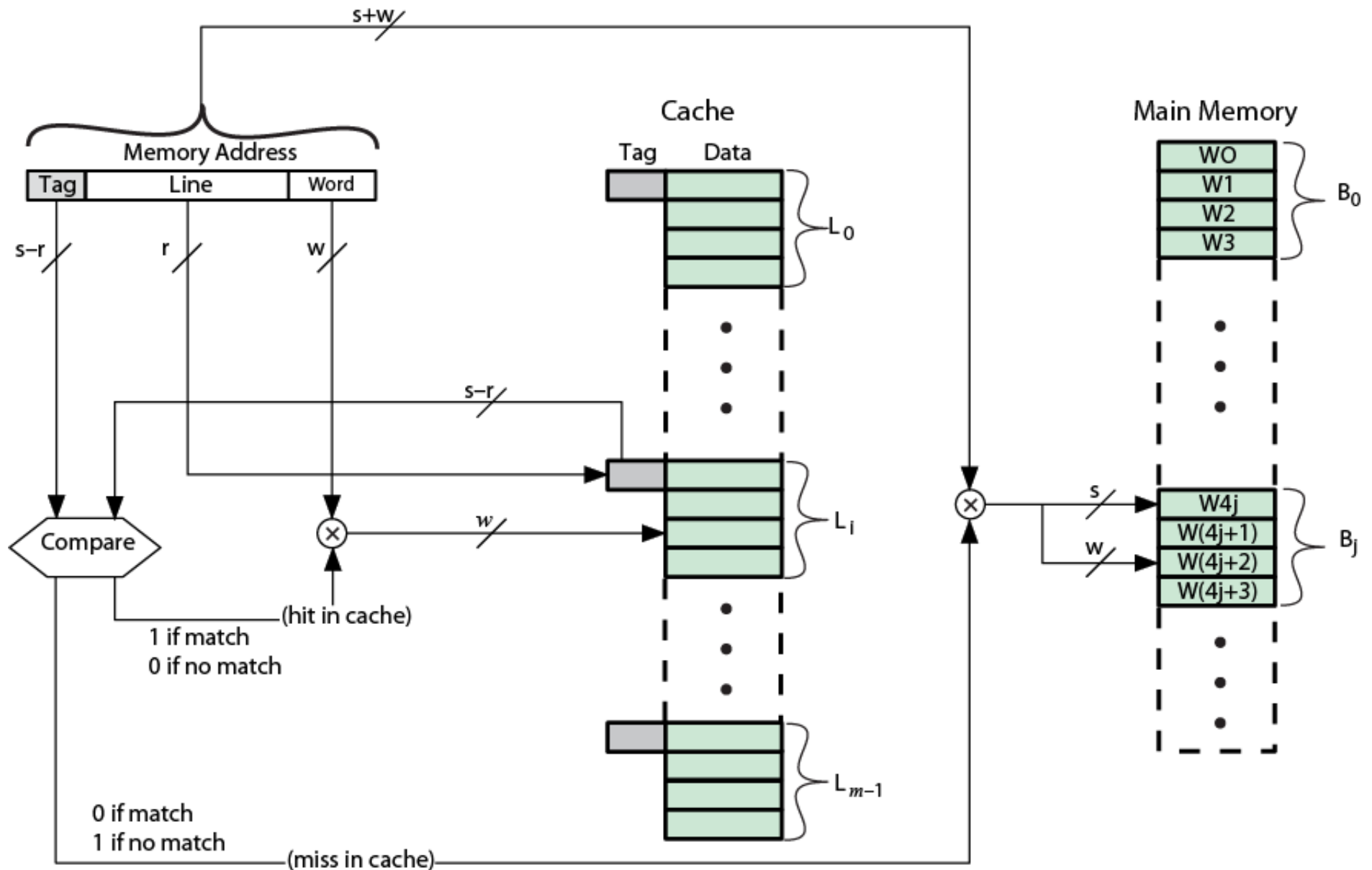
Main Memory

	Addr (hex)	Data
Block 0	000000	F1
	000001	F2
	000002	F3
	000003	F4
	000004	AB
	...	
Block 1	1B0004	11
	1B0005	12
	1B0006	13
	1B0007	14

Address = 1B0007 = 0001 1011 0000 0000 0000 0111

Tag= 0001 1011 Line = 0000 0000 0000 01, Word = 11,

Direct Mapping Cache Organization



Example: Direct Mapping Function

Cache memory = 64kByte
Block size = 4 bytes
Main memory = 16MBytes

Block = 4bytes
= 2^2 words

Cache = (64k/4) lines
= 16k lines of 4 bytes each
= $2^4 * 2^{10}$
= 2^{14} lines of 4 bytes

Main = (16M/4) blocks
= 4M blocks of 4 byte each
= $2^2 * 2^{10} * 2^{10}$
= 2^{22} blocks of 4 byte each

Main memory = 16M
= $2^4 * 2^{10} * 2^{10}$
= 2^{24} words

Each word is directly addressable by 24-bit address

Addressable units: 2^{24}

Address length: 24 bits

Block size: 2^2 words

Blocks in main memory: 2^{22}

Lines in cache: 2^{14}

s = 22

w = 2

r = 14

Size of tag (s-r): 8

- **Direct mapping** is a procedure used to assign each memory block in the main memory **to a particular line in the cache.**
- **If a line is already filled** with a memory block and a new block needs to be loaded, **then the old block is discarded from the cache.**

Direct Mapping: Summary

Addressable length = $(s + w)$ bits

w- no.of words in a block

Number of addressable units = 2^{s+w} bytes or words

Block size = line size = 2^w bytes or words

Number of blocks in main memory = $2^{s+w}/2^w = 2^s$

Number of lines in cache = $m = 2^r$ (r- slots/lines)

Size of cache = 2^{r+w} bytes or words

Size of tag $t = (s - r)$ bits where t is tag bits

Direct Mapping pros & cons

Advantages of direct mapping

- Here **only tag field is required to match** while searching word & hence it is **the fastest cache**.

Simple, Less expensive compared to associative cache mapping.

Fixed location for given block

Disadvantages of direct mapping

If a program accesses 2 blocks that map to the same line repeatedly, cache misses are very high – condition called **thrashing**

Swapping out a piece of a process just before that piece is needed.

Example

- Consider a machine with a byte addressable main memory of 2^{16} bytes and block size of 8 bytes. Assume that a direct mapped cache consisting of 32 lines is used with this machine. How is a 16-bit memory address divided into tag, line number, and byte number?
- **Answer:**
 - Block size = 8 bytes = 2^3 bytes
 - 3 bits are used for byte offset.
 - Cache lines = 32 = 2^5
 - 5 bits are used as index bits.
 - Main memory = 2^{16} bytes
 - Each word is directly addressable by 16-bit address
 - Remaining $16 - (5 + 3) = 8$ bits are used as tag bits.

Tag	Line	Bytes
8	5	3

Assignment

1. A 16MB main memory has 32KB cache with 8 bytes per line.
 - i) How many lines are there in the cache?
 - ii) Show how the main memory and cache memory is organized when the cache is direct- mapped.
 - iii) Show how the main memory address is partitioned.

2. A digital computer has a memory unit of 128K x 16 and a cache memory of 1K words. The cache uses direct mapping with a block size of four words.
 - How many bits are there in the tag, index, block and word fields of the address format?
 - How many blocks can the cache accommodate?

Summary:

Data has to be stored only in a specified cache location as per the mapping rule for direct mapping. **So it doesn't need replacement algorithm.**

Instead of storing total address information with data in cache **only part of address bits along with data are stored.**

Associative Mapping

Associative mapping scheme can overcome the problems of the direct mapping scheme

- A **main memory block can be loaded into any line of cache**

- Memory address is interpreted as **tag and word**

- Tag uniquely identifies block of memory

- Every line's tag is examined for a match

- Also need a Dirty and Valid bit

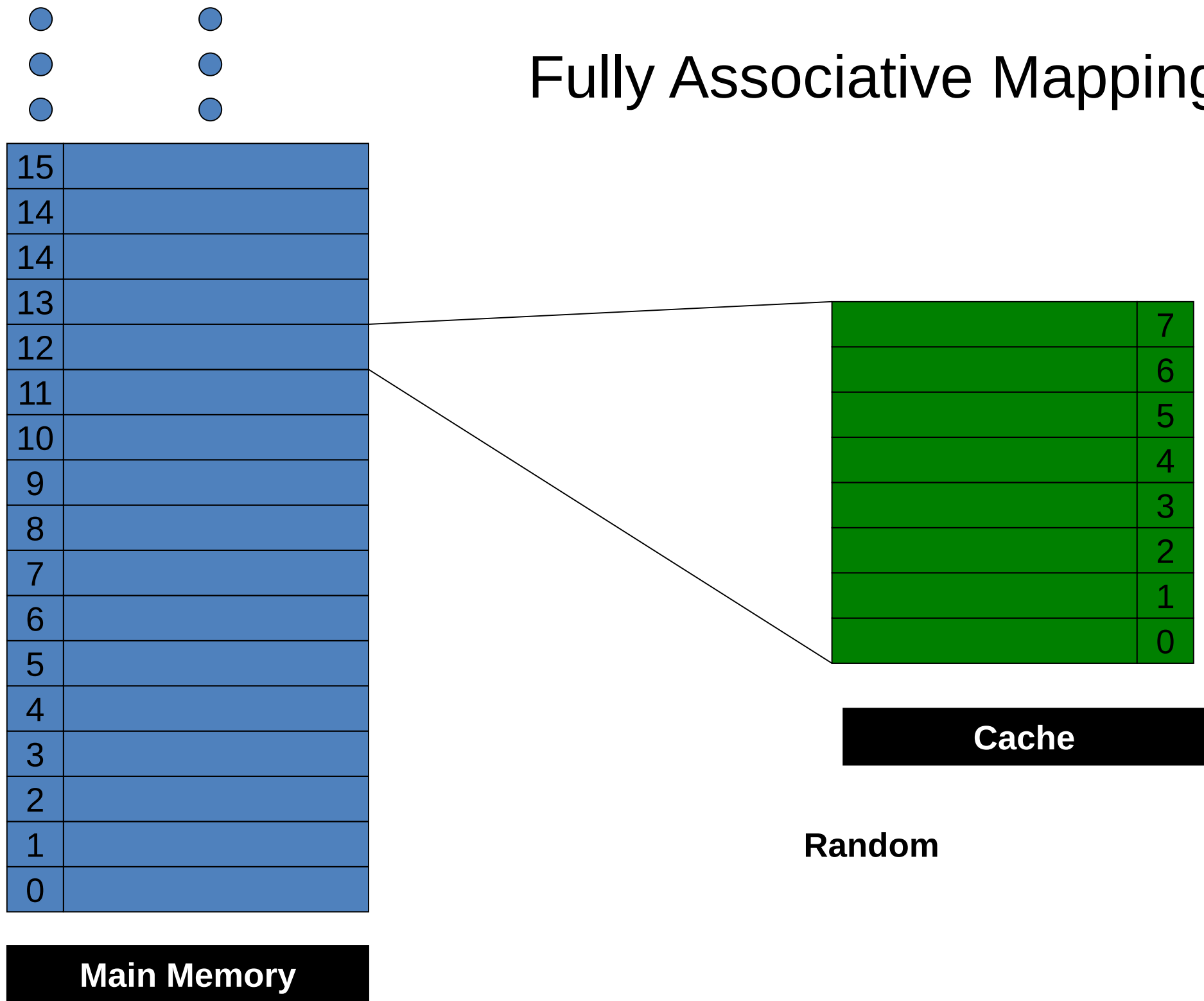
But **Cache searching gets expensive!**

- Ideally need circuitry that can simultaneously examine all tags for a match

- Lots of circuitry needed, hence high cost

Need **replacement policies(FIFO, LRU, Optimal replacement)** to avoid get thrown out of the cache

Fully Associative Mapping



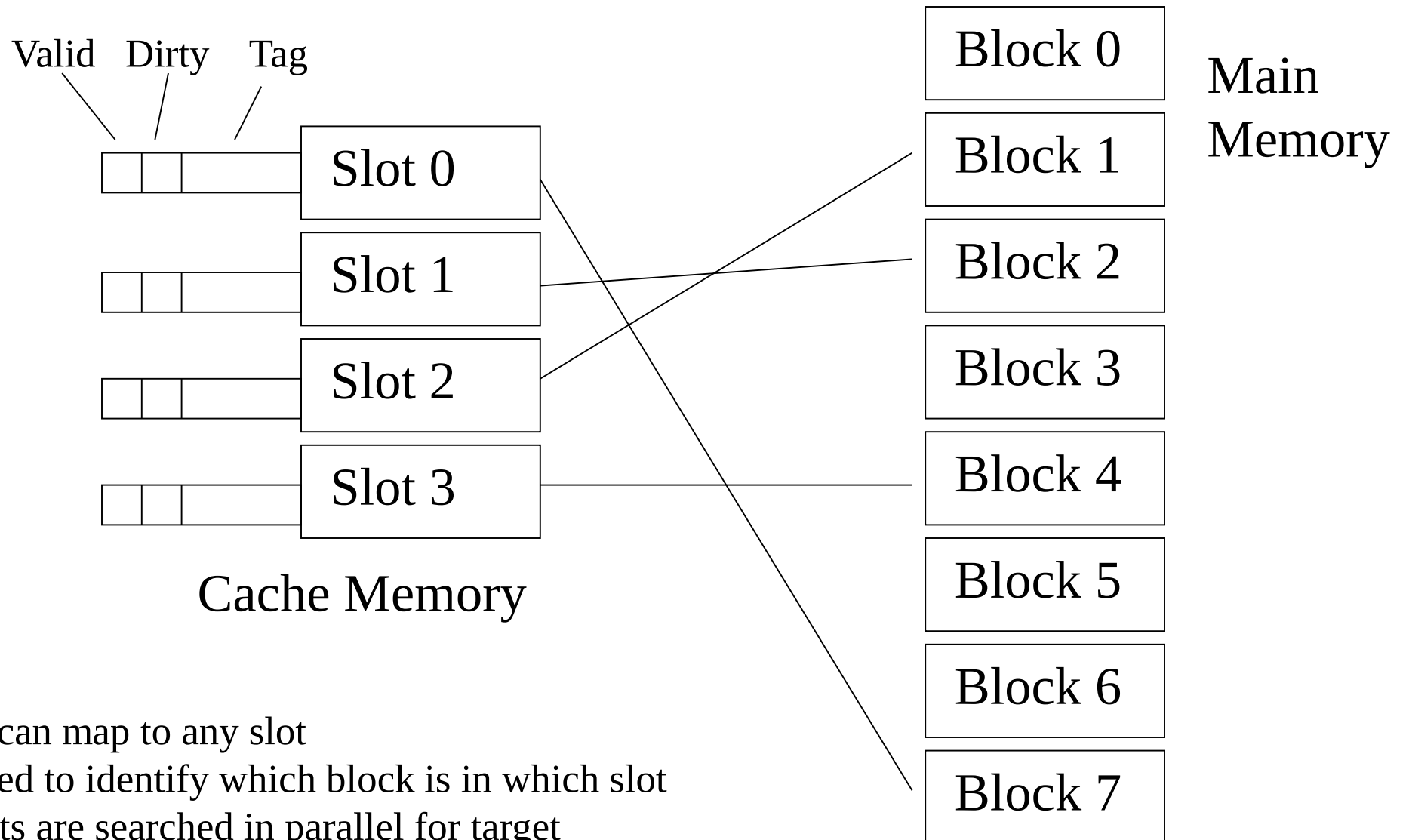
Associative Mapping

Address Structure

Tag 22 bit	Word 2 bit
------------	---------------

- Compare tag field with tag entry in cache to check for hit
- Least significant 2 bits of address identify the word

Associative Mapping Example



Block can map to any slot

Tag used to identify which block is in which slot

All slots are searched in parallel for target

Associative Mapping 64KB Cache Example

Tag 22 bit	Word 2 bit
------------	---------------

- Consider 16MB MM , 64KB CM with block/Line size of 4.
- 22 bit tag stored with each slot in the cache – no more bits for the slot line number needed since all tags searched are in parallel
- Compare tag field of a target memory address with tag entry in cache to **check for hit**
- Least significant 2 bits of address identify which word is required from the block, e.g.:

Address 24 bits: FFFFC = 1111 1111 1111 1111 1111 1100

Tag: Left 22 bits,

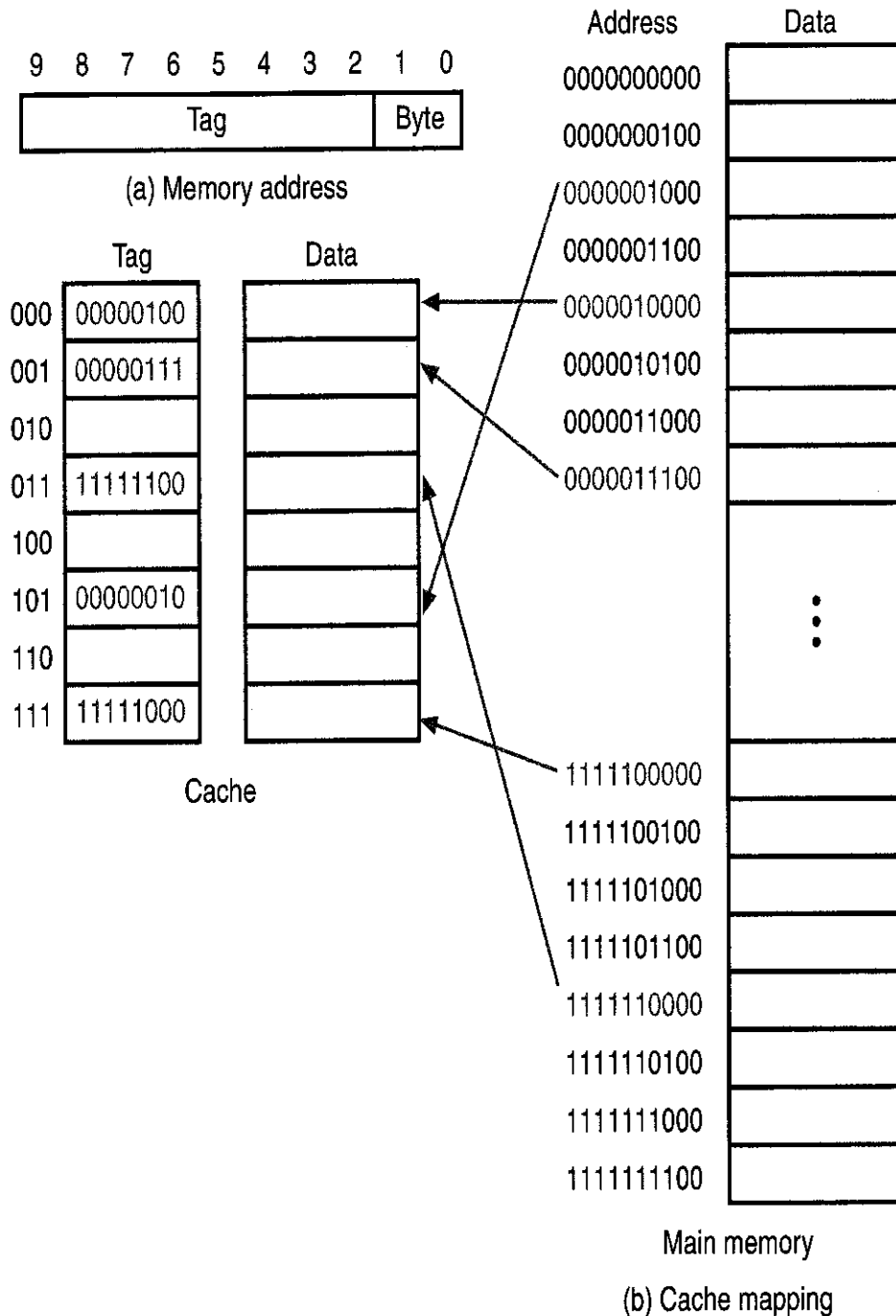
11 1111 1111 1111 1111 1111 = 3FFFFFF

Address: 16339C = 0001 0110 0011 0011 1001 1100

Tag: Left 22 bits,

00 0101 1000 1100 1110 0111 = 058CE7

Associative Cache Mapping

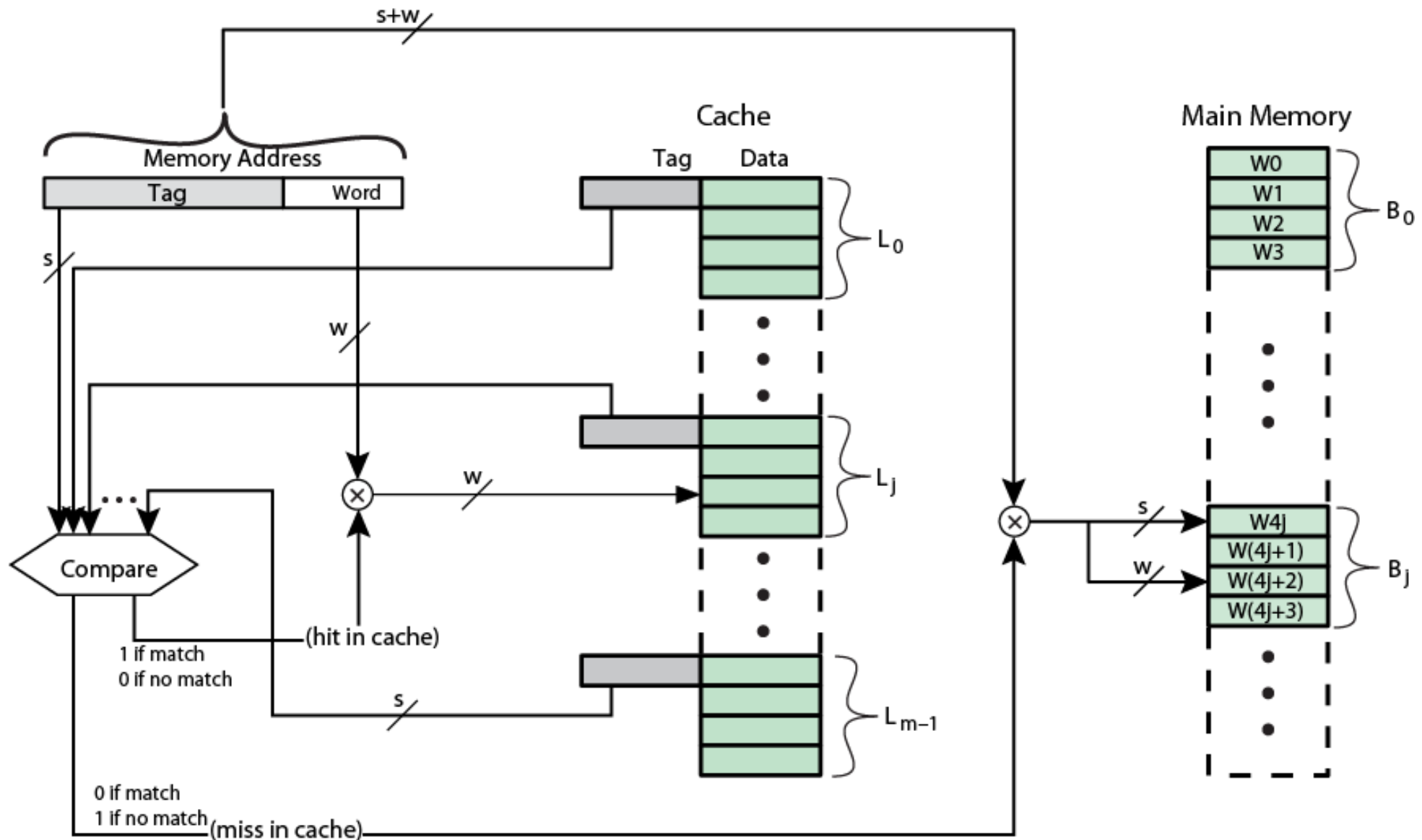


Any memory line can be placed into any location in the cache

When a request for data comes from the CPU, **the entire main memory tag address must be compared with all tag addresses presently residing in the cache**, to see if the requested data word is in the cache.

If not, a miss occurs and a new data line (with the requested word) is brought into the cache from main memory through replacement algorithms.

Associative Cache Organization



Example: Associative Mapping

Cache memory = 64kByte
Block size = 4 bytes
Main memory = 16MBytes

Block = 4bytes
= 2^2 words

Cache = (64k/4) lines
= 16k lines of 4 bytes each
= $2^4 * 2^{10}$
= 2^{14} lines of 4 bytes

Main = (16M/4) blocks
= 4M blocks of 4 byte each
= $2^2 * 2^{10} * 2^{10}$
= 2^{22} blocks of 4 byte each

Main memory = 16M
= $2^4 * 2^{10} * 2^{10}$
= 2^{24} words

Each word is directly addressable by 24-bit address

Addressable units: 2^{24}

Address length: 24 bits

Block size: 2^2 words

Blocks in main memory: 2^{22}

Lines in cache: 2^{14}

$s = 22$

$w = 2$

Size of tag: 22

Summary: Associative Mapping

- Any block location in Cache can store any block of primary memory
- Both the **address and data** of the memory word are stored.
- Mapping Table** is implemented in an associative memory (Content addressable memory)

This mapping method is also known as *fully associative cache*.

Advantages of associative mapping

Most flexible

- Easy to implement.

Disadvantages of associative mapping

- Expensive as it requires to store address along with the data
- Search time is more

Associative Mapping Summary

Address length = $(s + w)$ bits

Number of addressable units = 2^{s+w} words or bytes

Block size = line size = 2^w words or bytes

Number of blocks in main memory = $2^{s+w}/2^w = 2^s$

Size of tag = s bits

Set Associative Mapping

Compromise between associative and direct-mapped cache

Cache is **divided into a number of sets with each set containing a number of lines**

A given block maps to any line in a specific set

Use **direct-mapping** to determine which set in the cache corresponds to a set in main memory

Memory block could then be in any line of that set

e.g. K lines per set

K way associative mapping

A given block can be in one of K lines in a specific set

Much easier to simultaneously search one set than all lines

e.g. 2 lines per set

2 way associative mapping

A given block can be in either of 2 lines in a specific set

MM- 64 Bytes = 6 address lines

CM – 32 Bytes = 5 address lines

Block size- 4 Bytes = 2 bit address

Set size = 2 blocks (combined as a set) – 2 way set associative.

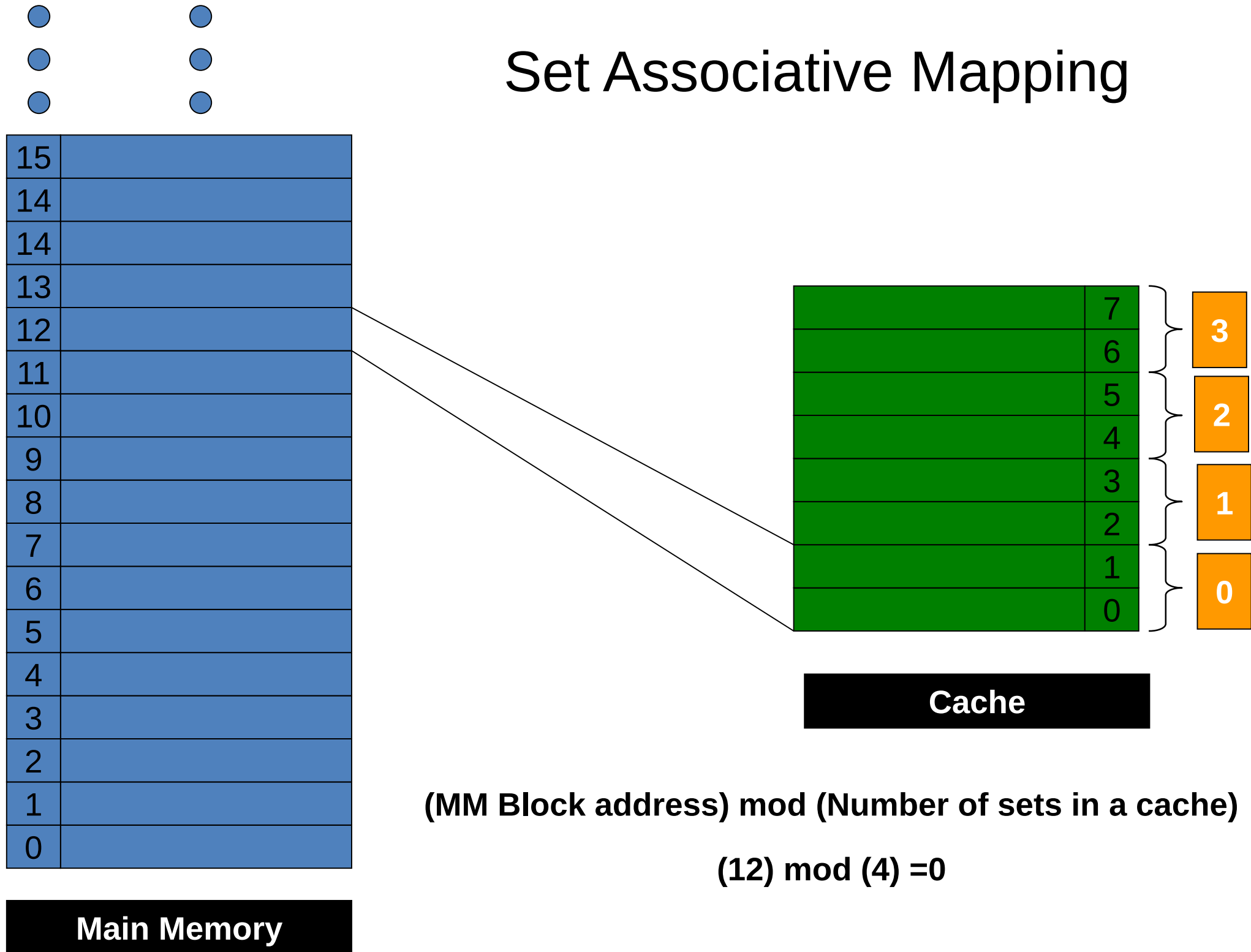
$32/4=8$ lines in cache

No. of sets = No. of lines/ Set size = $8/2=4$ sets

Physical address – 6 bits

Tag.No	Set. No.	Word
2	2	2

Set Associative Mapping



Set Associative Mapping

Address Structure – 24 bit address

Tag 9 bit	Set 13 bit	Word 2 bit
-----------	------------	------------

- Use set field to determine cache set to look in
- Compare tag field to see if we have a hit

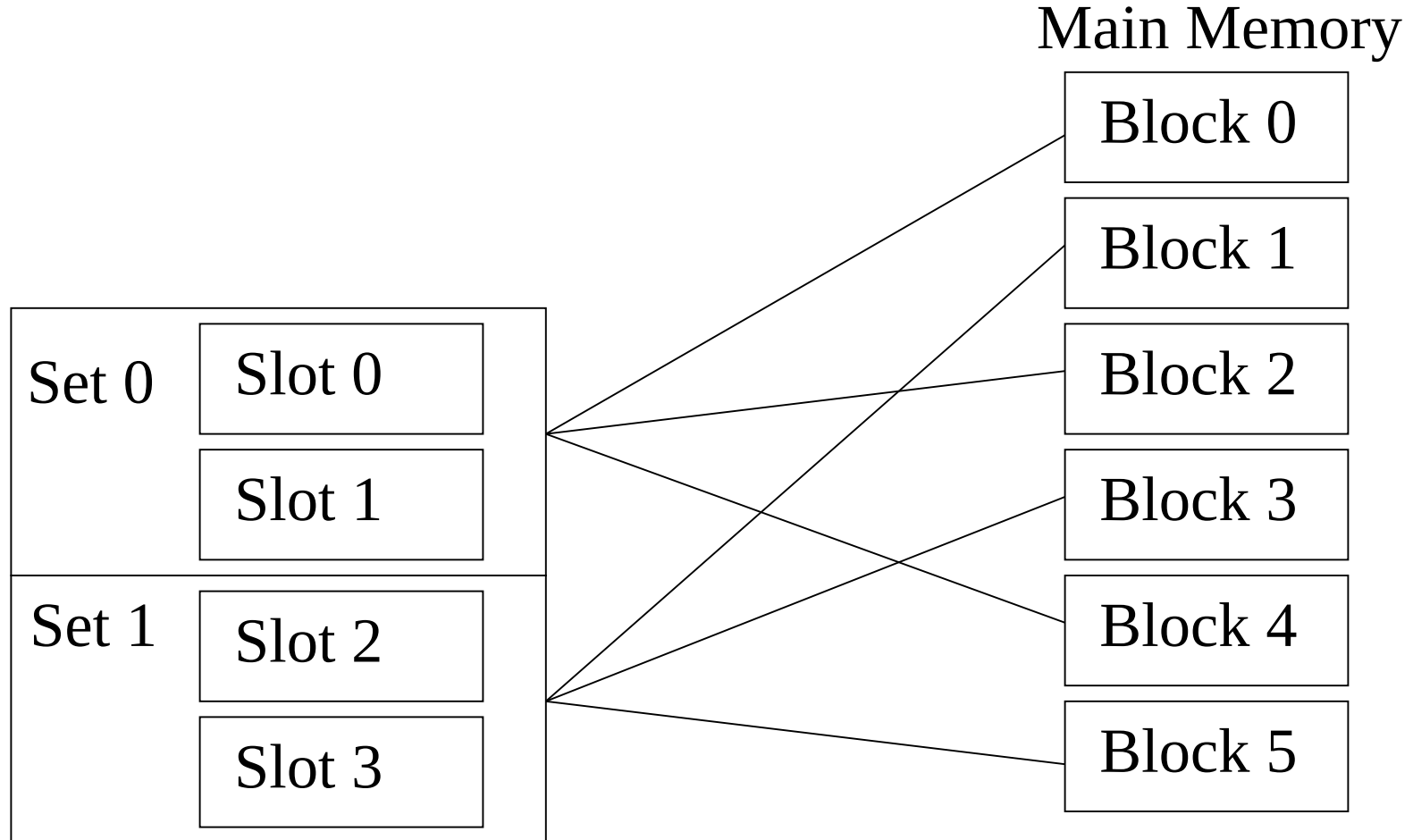
Set Associative Mapping

To compute cache set number:

$$\text{SetNum} = j \bmod v$$

j = main memory block number

v = number of sets in cache



Set Associative Mapping 64K Cache

Tag 9 bit	Set 13 bit	Word 2 bit
-----------	------------	------------

Given **64KB(2^{16})**cache, with a **line size of 4 bytes**, we have 16384 lines (2^{14}). Say that we decide to create 8192 sets ($2^{14} / 2^1 = 2^{13}$) where each set contains 2 lines. Then we need 13 bits to identify a set ($2^{13}=8192$). Main memory is 16MB.

Use set field to determine cache set to look in

Compare tag field of all slots in the set to see if we have a hit, e.g.:

Address = **16339C** = 0001 0110 00**11 0011 1001 1100**

Tag = 0 0010 1100 = 02C

Set = **0 1100 1110 0111** = 0CE7

Word = **00** = 0

Address = **008004** = 0000 0000 1**000 0000 0000 0100**

Tag = 0 0000 0001 = 001

Set = **0 0000 0000 0001** = 0001

Word = **00** = 0

Example: Two way set associative Mapping

Cache memory = 64kByte
Block size = 4 bytes
Main memory = 16MBytes
Each Set = 2 lines

Block = 4bytes
= 2^2 words

Set size = $2 * 4 = 8 = 2^3$ words

Cache = (64k/4) lines
= 16k lines of 4 bytes each
= $2^4 * 2^{10}$
= 2^{14} lines of 4 bytes

Cache = 64kByte
= (64k/8) sets
= 8k sets of 2 lines each
= $2^3 * 2^{10}$
= 2^{13} sets of 2 lines each

Main memory = (16M/4) blocks
= 4M blocks of 4 byte each
= $2^2 * 2^{10} * 2^{10}$
= 2^{22} blocks of 4 byte each

Main memory = 16M
= $2^4 * 2^{10} * 2^{10}$
= 2^{24} words

Each word is directly addressable by 24-bit address

Addressable units: 2^{24}

Address length: 24 bits

Block size: 2^2 words

Blocks in main memory: 2^{22}

Lines in set: 2

Number of sets: 2^{13}

Lines in cache: 2^{14}

$s = 22$

$w = 2$

$d = 13$

Size of tag (s-d): 9

Example: Set Associative Mapping

A set associative cache consists of 64 lines divided into four-line sets. Main memory contains 4K blocks of 128 words each. Show the format of memory addresses.

Each block contains 128 words.

$$\begin{aligned}\text{Block} &= 128 \text{ words.} \\ &= 2^7 \text{ words}\end{aligned}$$

Therefore, 7 bits are needed to identify the word within the block.

Cache = 64 lines / 4

Cache is divided into 16 sets of 4 lines each (2^4 sets).

Therefore, 4 bits are needed to identify the set number.

Main memory = 4K blocks of 128 words each. ($2^2 * 2^{10} = 2^{12}$ blocks)

Therefore, 12 bits are needed to specify the block (set + tag = 12 bits).

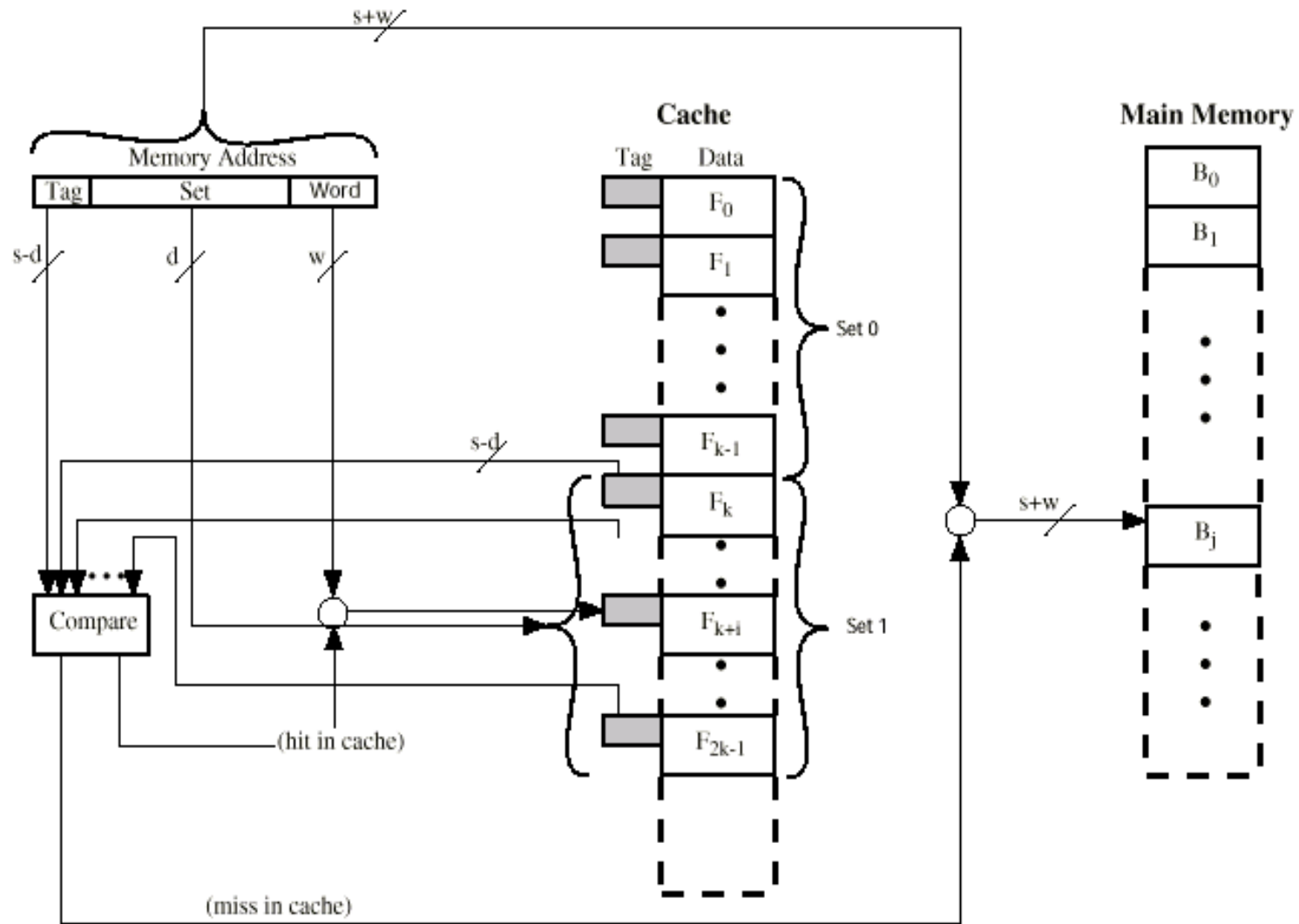
Tag length is 8 bits.

Main memory = 4K blocks of 128 words each. ($2^2 * 2^{10} * 2^7 = 2^{19}$ words)

Each word is directly addressable by 19-bit address.

TAG	SET	WORD
8	4	7

Two Way Set Associative Cache Organization



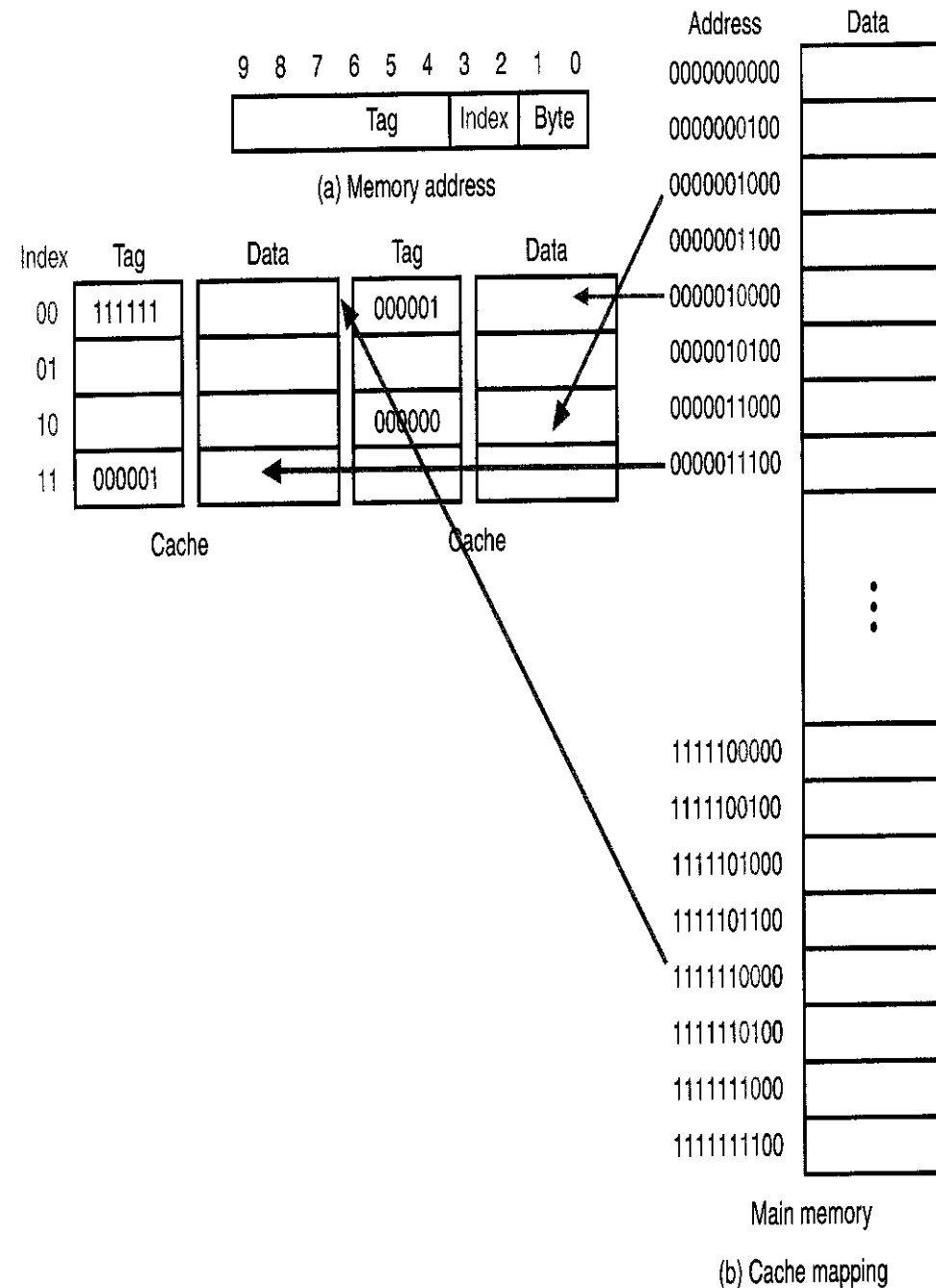
Two-way Set Associative Cache Mapping

A set associative cache is a compromise between direct mapped and fully associative cache approaches

Index(set) bits again specify cache (set) address

can have two lines of data per set (2-way set associative) with two different tag addresses or four lines of data per set (4-way set associative) with four different tag addresses

Example at left: 2 index bits $\rightarrow$ 4 sets with two-way set associative organization



Set Associative Mapping Summary

- Address length = $(s + w)$ bits
- Number of addressable units = 2^{s+w} words or bytes
- Block size = line size = 2^w words or bytes
- Number of blocks in main memory = 2^s
- Number of lines in set = k
- Number of sets = $v = 2^d$
- Number of lines in cache = $k v = k * 2^d$
- Size of tag = $(s - d)$ bits

Set-Associative Mapping

In *Set-Associative cache memory* two or more words can be stored under the **same index address**.

Every data word is stored along with its tag. The number of tag-data words under an index is said to form a text.

Advantages of Set-Associative mapping

- Has **highest hit-ratio** compared to other two cache memory mapping. Thus its **performance is considerably better**.
- No. of comparators is reduced.

Disadvantages of Set-Associative mapping

- Very expensive - Cost increases as the set size increases.

Example

Consider a machine with a byte addressable main memory of 2^{16} bytes and block size of 8 bytes. Assume that a direct mapped cache consisting of 32 lines is used with this machine. How is a 16-bit memory address divided into tag, line number, and byte number?

Answer:

Block size = 8 bytes = 2^3 bytes

3 bits are used for byte offset.

Cache lines = 32 = 2^5

5 bits are used as index bits.

Main memory = 2^{16} bytes

Each word is directly addressable by 16-bit address

Remaining $16 - (5 + 3) = 8$ bits are used as tag bits.

Tag	Line	Bytes
8	5	3

Assignment

1. A 16MB main memory has 32KB cache with 8 bytes per line.
 - i) How many lines are there in the cache?
 - ii) Show how the main memory and cache memory is organized when the cache is direct- mapped.
 - iii) Show how the main memory address is partitioned.

2. A digital computer has a memory unit of 128K x 8 and a cache memory of 1K words. The cache uses direct mapping with a block size of four words.

How many bits are there in the tag, index, block and word fields of the address format?

How many blocks can the cache accommodate?

Replacement Policy - Direct mapping

The replacement policy is the technique used to **determine which line in the cache should be thrown out when we want to bring a new block to cache from main memory**

Direct mapping

Replacement policy is not required

Each block only maps to one line; Replace that line

Replacement Algorithms-Associative & Set Associative

Least Recently used (LRU)

e.g. in 2 way set associative, 2 blocks LRU has to be replaced.

For each slot, have an extra bit, **USE**. Set to 1 when accessed, set all others to 0.

First in first out (FIFO)

Replace block that has been in cache longest

Easy to implement as a circular buffer

Least frequently used

Replace block which had fewest hits

Need a counter to sum number of hits

Random

Almost as good as LFU and simple to implement

Write Policy

- Must not overwrite a cache block unless main memory is up to date
- Multiple CPUs may have individual caches
- I/O may address main memory directly
- **Write through**
 - All writes go to main memory as well as cache
 - Multiple CPUs can monitor main memory traffic to keep local (to CPU) cache up to date
 - Lots of traffic
 - Slows down writes

Write Policy

- **Write back**
 - Updates initially made in cache only.
 - Update bit for cache slot is set when update occurs
 - If block is to be replaced, write to main memory only if update bit is set
 - Other caches get out of sync.
 - I/O must access main memory through cache

Multilevel Caches

- High logic density enables caches on chip
 - Faster than bus access
 - Frees bus for other transfers
- Common to use both on and off chip cache
 - L1 on chip, L2 off chip in static RAM
 - L2 access much faster than DRAM or ROM
 - L2 often uses separate data path
 - L2 may now be on chip
 - Resulting in L3 cache
 - Bus access or now on chip...

Unified v Split Caches

- One cache for data and instructions or two, one for data and one for instructions
- Advantages of unified cache
 - Higher hit rate
 - Balances load of instruction and data fetch
 - Only one cache to design & implement
- Advantages of split cache
 - Eliminates cache contention between instruction fetch/decode unit and execution unit